Linear Regression Demo (Synthetic Data)

Article ke saare steps ** data** par:

- **Part A:** Simple Linear Regression (Hours → Scores, 25 rows)
- **Part B:** Multiple Linear Regression (petrol consumption, 48 rows, 4 features)

Data nakli (randomly generated) hai, seed fixed hai isliye har run mein same results.

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
SEED = 42
rng = np.random.default_rng(SEED)

def evaluate(y_true, y_hat, name=""):
    mae = mean_absolute_error(y_true, y_hat); mse = mean_squared_error(y_true, y_hat)
    rmse = np.sqrt(mse); r2 = r2_score(y_true, y_hat)
    print(f"--- {name} ---")
    print(f"Mean absolute error     : {mae:.2f}")
    print(f"Mean squared error      : {mse:.2f}")
    print(f"Root mean squared error : {rmse:.2f}")
    print(f"R2                      : {r2:.4f}")
    return mae, mse, rmse, r2
```

Part A — Simple Linear Regression

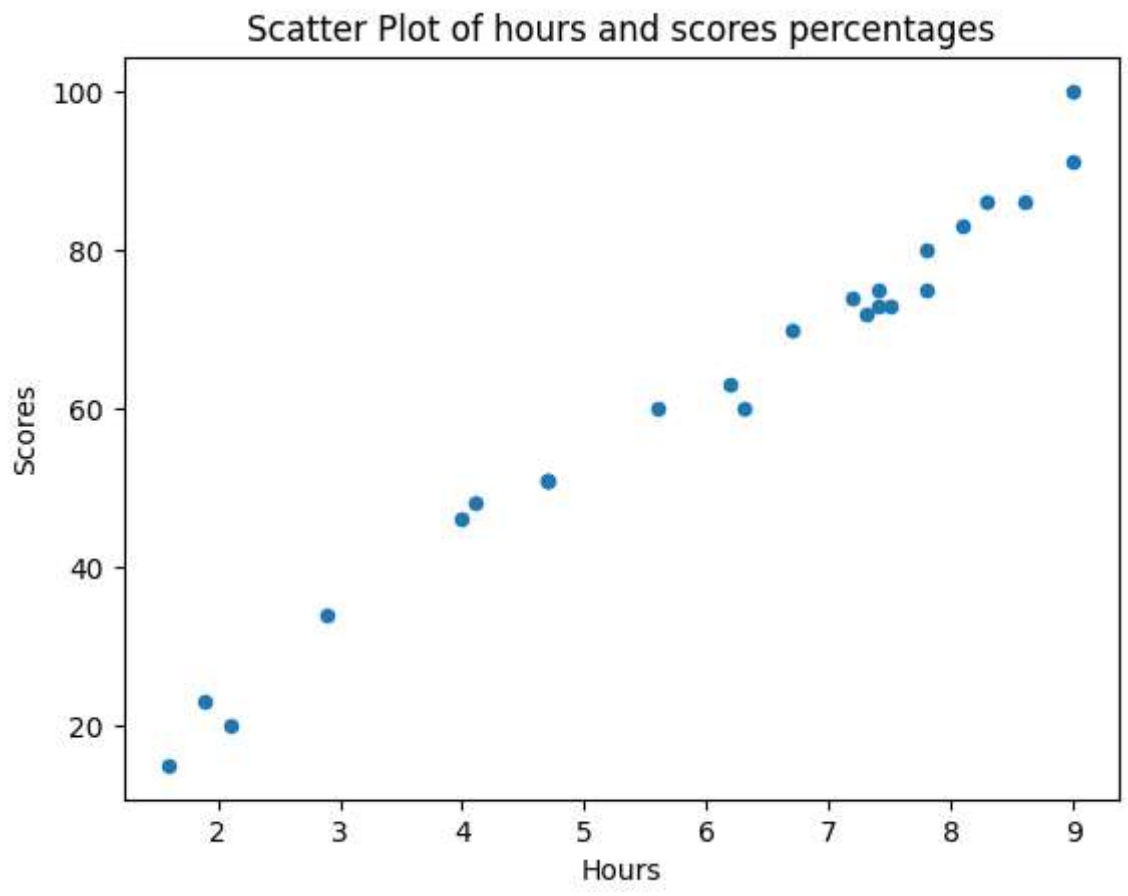
A1. Demo data banana

```
hours = np.round(rng.uniform(1.1, 9.2, 25), 1)
scores = np.clip(np.round(9.7 * hours + 2.8 + rng.normal(0, 4.5, 25)), 0, 100).astype(int)
df = pd.DataFrame({"Hours": hours, "Scores": scores})
print(df.head())
print("Shape:", df.shape)
```

```
   Hours  Scores
0    7.4      73
1    4.7      51
2    8.1      83
3    6.7      70
4    1.9      23
Shape: (25, 2)
```

A2. Exploratory Data Analysis

```
df.plot.scatter(x="Hours", y="Scores", title="Scatter Plot of hours and scores percentages")
plt.show()
print(df.corr())
print(df.describe())
```




```
            Hours     Scores
Hours    1.000000  0.990127
Scores   0.990127  1.000000
           Hours      Scores
count  25.00000   25.000000
mean    6.03600   62.400000
std     2.27905   22.535158
min     1.60000   15.000000
25%     4.70000   51.000000
50%     6.70000   70.000000
75%     7.80000   75.000000
max     9.00000  100.000000
```

A3. Preprocessing & Train/Test split

```
y = df["Scores"].values.reshape(-1, 1)
X = df["Hours"].values.reshape(-1, 1)
print(X.shape, y.shape)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)
print("Train:", X_train.shape, "Test:", X_test.shape)
```

```
(25, 1) (25, 1)
Train: (20, 1) Test: (5, 1)
```

A4. Training

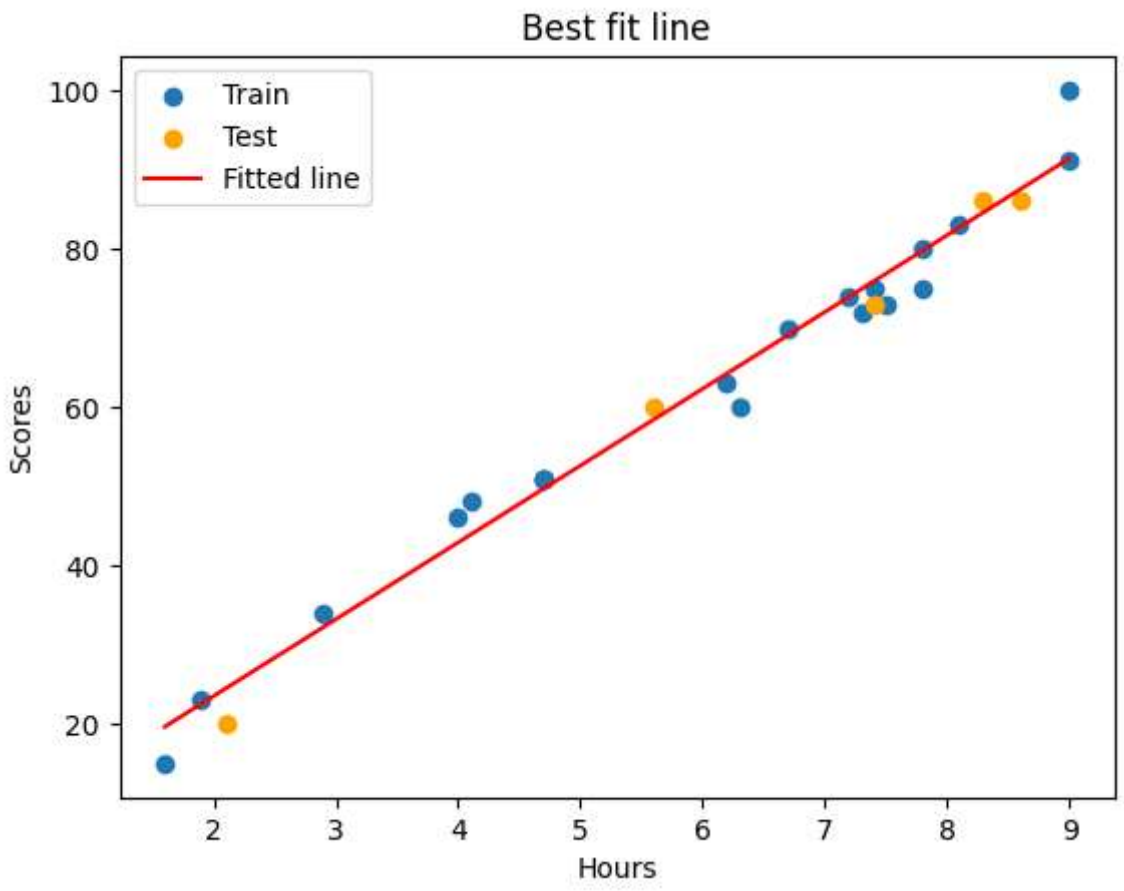
```
regressor = LinearRegression()
regressor.fit(X_train, y_train)
print("Intercept:", regressor.intercept_)
print("Slope    :", regressor.coef_)
print(f"score = {regressor.coef_[0][0]:.4f} * hours + {regressor.intercept_[0]:.4f}")
print("Prediction for 9.5 hours:", regressor.predict([[9.5]]))
```

```
Intercept: [4.14561231]
Slope    : [[9.68955218]]
score = 9.6896 * hours + 4.1456
Prediction for 9.5 hours: [[96.19635799]]
```

A5. Predictions on test data

```
y_pred = regressor.predict(X_test)
print(pd.DataFrame({"Actual": y_test.squeeze(), "Predicted": y_pred.squeeze()}))
xs = np.linspace(X.min(), X.max(), 50).reshape(-1, 1)
plt.scatter(X_train, y_train, label="Train"); plt.scatter(X_test, y_test, label="Test", color="orange")
plt.plot(xs, regressor.predict(xs), color="red", label="Fitted line")
plt.xlabel("Hours"); plt.ylabel("Scores"); plt.legend(); plt.title("Best fit line"); plt.show()
```

```
   Actual  Predicted
0      20  24.493672
1      60  58.407104
2      73  75.848298
3      86  84.568895
4      86  87.475761
```



A6. Evaluation

```
res_simple = evaluate(y_test, y_pred, "Simple LR (test)")
print("Train R2:", round(regressor.score(X_train, y_train), 4), "| Test R2:", round(regressor.score(X_test, y_test), 4)
```

```
--- Simple LR (test) ---
Mean absolute error     : 2.37
Mean squared error      : 7.01
Root mean squared error : 2.65
R2                      : 0.9883
Train R2: 0.9774 | Test R2: 0.9883
```

Part B — Multiple Linear Regression

B1. Demo data banana

```
n = 48
tax = np.round(rng.uniform(5, 10, n) * 2) / 2
income = rng.normal(4242, 574, n).round()
highways = rng.gamma(2.5, 2200, n).round().clip(431, 17782)
licence = np.clip(rng.normal(0.57, 0.06, n), 0.45, 0.72).round(3)
consumption = (361 - 0.0565*income - 0.0044*highways + 1347*licence - 37*tax + rng.normal(0, 55, n)).round()
df2 = pd.DataFrame({"Petrol_tax": tax, "Average_income": income, "Paved_Highways": highways,
                    "Population_Driver_licence(%)": licence, "Petrol_Consumption": consumption})
print(df2.head()); print("Shape:", df2.shape)
```

```
   Petrol_tax  Average_income  Paved_Highways  Population_Driver_licence(%)  \
0         9.0          4471.0          3047.0                         0.703
1         8.5          3722.0          9119.0                         0.538
2         8.5          4025.0          4271.0                         0.517
3         9.0          4988.0          1140.0                         0.564
4         7.5          4038.0          4595.0                         0.465

   Petrol_Consumption
0               723.0
1               524.0
2               481.0
3               495.0
4               448.0
Shape: (48, 5)
```

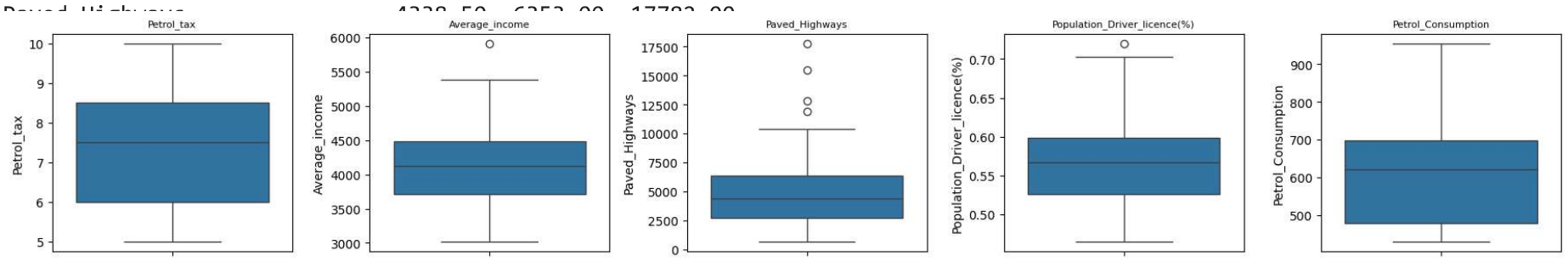
B2. EDA

```
print(df2.describe().round(2).T)
variables = ["Petrol_tax", "Average_income", "Paved_Highways", "Population_Driver_licence(%)"]
fig, axes = plt.subplots(2, 2, figsize=(11, 8))
for ax, var in zip(axes.ravel(), variables):
    sns.regplot(x=var, y="Petrol_Consumption", data=df2, ax=ax).set(title=f"{var} vs Petrol_Consumption")
plt.tight_layout(); plt.show()
plt.figure(figsize=(7, 5))
sns.heatmap(df2.corr(), annot=True, fmt=".2f", cmap="coolwarm").set(title="Heat map - Pearson Correlations")
plt.show()
```

```
                           count     mean      std      min      25%  \
Petrol_tax                  48.0     7.31     1.32     5.00     6.00
Average_income              48.0  4199.71   584.20  3018.00  3719.00
Paved_Highways              48.0  5210.46  3663.71   646.00  2711.50
Population_Driver_licence(%)48.0     0.57     0.06     0.46     0.53
```

B3. Outlier check (boxplots)

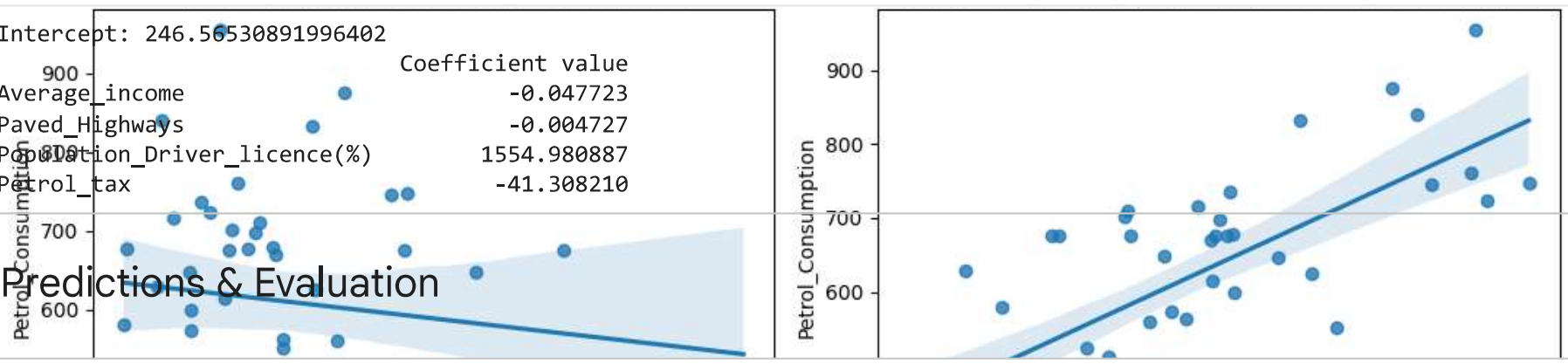
```
fig, axes = plt.subplots(1, 5, figsize=(18, 3))
for ax, c in zip(axes, df2.columns):
    sns.boxplot(y=df2[c], ax=ax); ax.set_title(c, fontsize=8)
plt.tight_layout(); plt.show()
```



B4. Train multivariate model

```
y = df2["Petrol_Consumption"]
X = df2[["Average_income", "Paved_Highways", "Population_Driver_licence(%)", "Petrol_tax"]]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)
regressor = LinearRegression().fit(X_train, y_train)
print("Intercept:", regressor.intercept_)
print(pd.DataFrame(regressor.coef_, index=X.columns, columns=["Coefficient value"]))
```

```
Intercept: 246.56530891996402
                            Coefficient value
Average_income                      -0.047723
Paved_Highways                      -0.004727
Population_Driver_licence(%)      1554.980887
Petrol_tax                         -41.308210
```



B5. Predictions & Evaluation

```
y_pred = regressor.predict(X_test)
print(pd.DataFrame({"Actual": y_test, "Predicted": y_pred}))
res_multi = evaluate(y_test, y_pred, "Multiple LR (test)")
r2_manual = 1 - sum((y_test - y_pred)**2) / sum((y_test - y_test.mean())**2)
print("R2 manual:", r2_manual)
print("Train R2:", round(regressor.score(X_train, y_train), 4), "| Test R2:", round(regressor.score(X_test, y_test), 4)
```

```
    Actual   Predicted
27   625.0  615.651061
40   552.0  673.069601
26   679.0  598.222435
43   449.0  457.094147
24   716.0  668.237460
```

